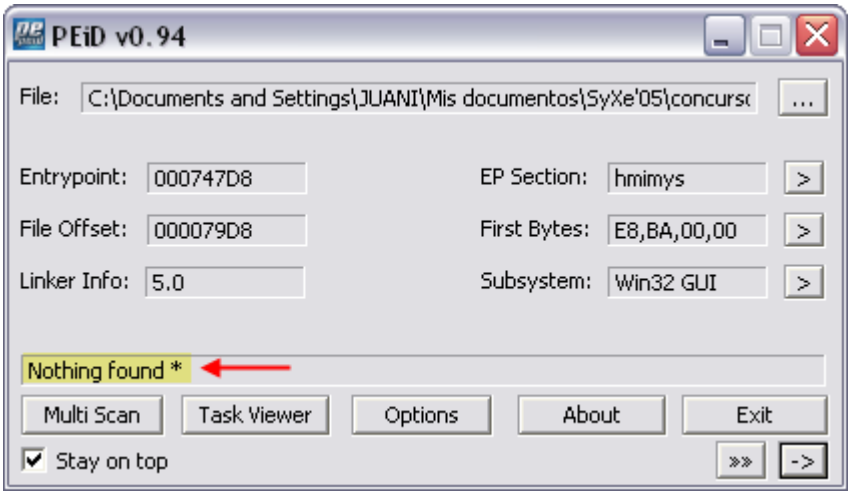
Solución al Concurso 106 nivel 3

Programa:	Hmimys Packer v1,0 UnPackMe
Protección:	Empaquetado con Hmimys 1.0
Objetivos:	Desempakar
URL descarga:	FTP de Ricardo
Dificultad:	Nula
Herramientas:	OllyDbg v1.10 + PEiD v0.94 + ImpREC v1.6
Cracker:	SyXe'05 .::[CracksLatinoS]::. Fecha: 27-10-06

Un saludo a todos CracksLatinoS!

Después de mucho tiempo sin escribir nada volvemos otra vez y en este caso para solucionar un concurso, ante todso pedir disculpas por no aparecer por la lista, gracias a mi hermano estoy esceribirnfo este tute para la lista que lo debo del pasado nivel, asi que veamos que hay xD, con PEiD 0.94 vemos:



hmmmmm.. no lo detecta en ninguno de los tres modos que ofrece:

1.- Normal Scan
2.- Deep Scan
3.- Hardcode Scan

Obteniendo los mismos resultados con cualquiera de los tres, lo siguiente será ver qué nos dice otra excelente herramienta programada por el amigo RDG llamada RDG Packer v0.64 Beta R-1, En modo M-B no analiza un carajo, pero seamos plomos y sigamos, picamos al modo M-A que es el primero que aparece aunque yo lo tengo configurado por defecto para que analice de entrada siempre con el modo M-B, que me enrollo de masiado, Nodo M-A y vemos esto:

Bueno no atinó pero ya al menos dijo algo, lo siguiente que haremos será cerrar estos dos analizadores y abrirnos un OllyIce, que tengo pelao de plugins, únicamente tiene estos:

1.- CommandBar 3.10.109c de Oleg Yushuk y modificado por Gigapede
2.- HideOD de ????
3.- OllyDump2.21.108 de Gigapede
4.- OllyScript 0,92 de SHaG

Dicho esto empecemos, si cargamos el UnPackMe nos llevamos una ingrata sorpresa ya que no aparece código ¿?

Address	Hex dump	Disassembly		Comment
3EE938B9	0000	add	[eax], al	
3EE938BB	0000	add	[eax], al	
3EE938BD	0000	add	[eax], al	
3EE938BF	0000	add	[eax], al	
3EE938C1	0000	add	[eax], al	
3EE938C3	0000	add	[eax], al	
3EE938C5	0000	add	[eax], al	
3EE938C7	0000	add	[eax], al	
3EE938C9	0000	add	[eax], al	

Tras darle vueltas al asunto descubrí que era por culpa de la ecepción:

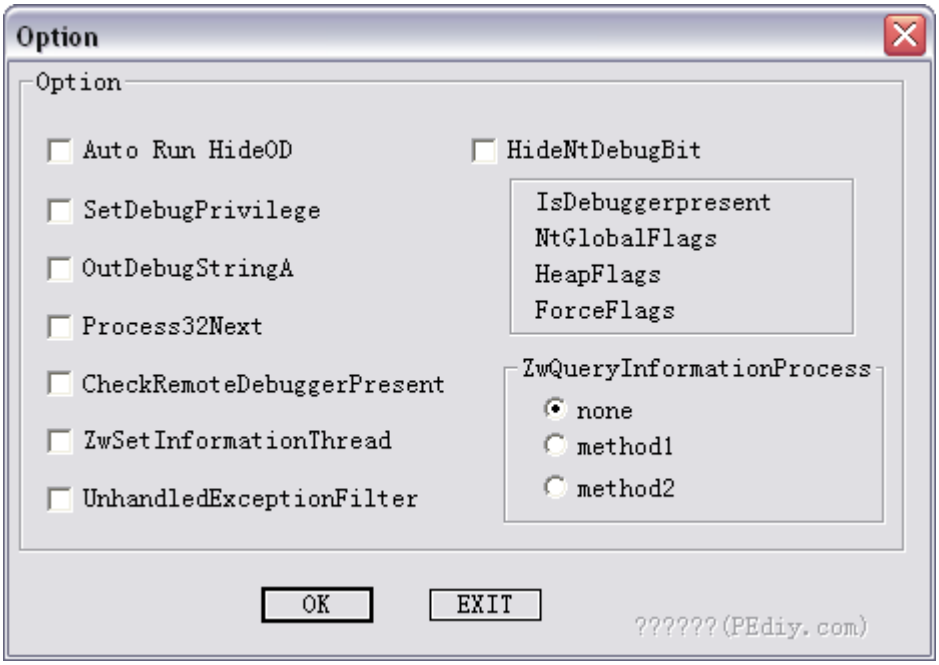
Que debe estar el chech marcado, y entonces sí carga bien el código ;)

Address	Hex dump	Disassembly		Comment
004747D8	E8 BA000000	call	00474897	
004747DD	0300	add	eax, [eax]	
004747DF	0000	add	[eax], al	
004747E1	00C0	add	al, al	
004747E3	06	push	es	
004747E4	0000	add	[eax], al	
004747E6	1040 00	adc	[eax], al	
004747E9	5D	pop	ebp	
004747EA	6302	arpl	[edx], ax	
004747EC	005A 51	add	[edx+51], bl	
004747EF	47	inc	edi	

Y si adeemás nos fijamos antes cargaba el código del unpackme a partrir de :3EE938B9 mientras que ahora lo carga a partir de :4747D8 como debe de ser.

Pasado este primer semi-escollo vamos al grano, ando corto de tiempo y como sabés estoy en casa de mi hermano mayor y encima en su ordenata, jeje.. pero bueno por algo somos hermanos no?

De momento el plugin HideOD lo tengo así configurado:



Apreto F9 y akl cunpackme corre, perfecto, recordemos que en la ventana de EXCEPTIONS tenemos únicamente marcada "MEMORY AVCCESS VIOLATION" así que para descubrir el OEP no nos vale el método de Ricardo de las excepciones, intentaremos algo más simple todavía, como es un crackme y con más bien poco código traceamos a mano desde el EntryPoint, pero OJO!! con F7 sino nos manda al carajo, entramos en el primer call y vemos:

Address	Hex dump	Disassembly		Comment
00474897	5E	pop	esi	UnPackMe.004747DD
00474898	83C6 64	add	esi, 64	
0047489B	AD	lods	dword ptr [esi]	
0047489C	50	push	eax	
0047489D	AD	lods	dword ptr [esi]	
0047489E	50	push	eax	
0047489F	83EE 6C	sub	esi, 6C	
004748A2	AD	lods	dword ptr [esi]	
004748A3	50	push	eax	
004748A4	AD	lods	dword ptr [esi]	
004748A5	50	push	eax	
004748A6	AD	lods	dword ptr [esi]	
004748A7	50	push	eax	
004748A8	AD	lods	dword ptr [esi]	
004748A9	50	push	eax	
004748AA	AD	lods	dword ptr [esi]	
004748AB	50	push	eax	
004748AC	E8 E7070000	call	00475098	
004748B1	AD	lods	dword ptr [esi]	
004748B2	8BDE	mov	ebx, esi	
004748B4	8BF0	mov	esi, eax	
004748B6	83C3 44	add	ebx, 44	
004748B9	AD	lods	dword ptr [esi]	
004748BA	85C0	test	eax, eax	
004748BC	74 32	je	short 004748F0	
004748BE	8BF8	mov	edi, eax	

Si la traceamos enseguida vemos cositas interesantes, hmmmm..

Address	Hex dump	Disassembly		Comment
00474897	5E	pop	esi	
00474898	83C6 64	add	esi, 64	
0047489B	AD	lods	dword ptr [esi]	
0047489C	50	push	eax	kernel32.VirtualFree
0047489D	AD	lods	dword ptr [esi]	
0047489E	50	push	eax	

Ya vemos una API llamada VirtualFree utilizada para liberar bloques de memoria solicitados con VirtualAlloc una vez ya no son necesarios, sigamos a aver...

Address	Hex dump	Disassembly		Comment
00474897	5E	pop	esi	
00474898	83C6 64	add	esi, 64	
0047489B	AD	lods	dword ptr [esi]	
0047489C	50	push	eax	
0047489D	AD	lods	dword ptr [esi]	
0047489E	50	push	eax	kernel32.VirtualAlloc
0047489F	83EE 6C	sub	esi, 6C	
004748A2	AD	lods	dword ptr [esi]	

Mira por donde ahí apareció, y ya no vemos ninguna más, entremos en esta llamada a ver si hay algo:

Address	Hex dump	Disassembly		Comment
004748A9	50	push	eax	
004748AA	AD	lods	dword ptr [esi]	
004748AB	50	push	eax	
004748AC	E8 E7070000	call	00475098	
004748B1	AD	lods	dword ptr [esi]	

Un par de movimientos para saltar a la API VirtualAlloc:

Address	Hex dump	Disassembly		Comment
004750D9	50	push	eax	
004750DA	6A 00	push	0	
004750DC	FF55 1C	call	[ebp+1C]	kernel32.VirtualAlloc
004750DF	85C0	test	eax, eax	
004750E1	8945 FC	mov	[ebp-4], eax	
004750E4	75 04	jnz	short 004750EA	

A la salida si todo fue bien tendremos la dirección de inicio del bloque asignado por el sistema, en este caso nos da:

Registers (FPU)	
EAX 00140000	←
ECX 77E5AC6F	kernel32.77E5AC6F
EDX 7FFE0304	
EBX 0002635D	
ESP 0012FF88	

Allí estará la nueva sección poedida y por supuesto asignada como podemos ver con un simple ALT+M para entrar a la ventanita de memoria y comprobarlo:

Address	Size	Owner	Section	Contains	Type	Access		Initial	Mapped as
00010000	00001000				Priv	RW		RW	
00020000	00001000				Priv	RW		RW	
0012C000	00001000				Priv	RW	Guar	RW	
0012D000	00003000			stack of ma	Priv	RW	Guar	RW	
00130000	00001000				Map	R		R	
00140000	00008000				Priv	RW		RW	
00150000	00006000				Priv	RW		RW	
00250000	00006000				Priv	RW		RW	
00260000	00001000				Map	RW		RW	
00270000	00016000				Map	R		R	\Device\HarddiskVolume1\WINDOWS\sy
00290000	00034000				Map	R		R	\Device\HarddiskVolume1\WINDOWS\sy
002D0000	00041000				Map	R		R	\Device\HarddiskVolume1\WINDOWS\sy
00320000	00006000				Map	R		R	\Device\HarddiskVolume1\WINDOWS\sy

Entramosen la siguiente llamada y vemos que comienza a desencriptar la seccion de código aquí:

Address	Hex dump	Disassembly		Comment
00474E6A	8B4D 08	mov	ecx, [ebp+8]	
00474E6D	FF45 08	inc	dword ptr [ebp+8]	
00474E70	8AD8	mov	bl, al	
00474E72	8B45 18	mov	eax, [ebp+18]	
00474E75	83FF 04	cmp	edi, 4	
00474E78	881C08	mov	[eax+ecx], bl	escribe seccion .code
00474E7B	7D 09	jge	short 00474E86	
00474E7D	8365 F8 00	and	dword ptr [ebp-8], 0	
00474E81	E9 E1010000	jmp	00475067	
00474E86	83FF 0A	cmp	edi, 0A	
00474E89	7D 0B	jge	short 00474E96	
00474E8B	83EF 03	sub	edi, 3	
00474E8E	897D F8	mov	[ebp-8], edi	
00474E91	E9 D1010000	jmp	00475067	
00474E96	83EF 06	sub	edi, 6	
00474E99	EB F3	jmp	short 00474E8E	

en ese:

```
00474E78    881C08          mov     [eax+ecx], bl
```

Comienza a escribier en ella como se puede ver en los registros ;)

Registers (FPU)
EAX 00401000 UnPackMe.00401000
ECX 00000000
EDX 0012FF2C ASCII "nQG"
EBX 00026356

Como EAX==401000 y ECX==00000000 tenemos que:

[EAX+ECX]==401000

El inicio de la sección de código:

Address	Size	Owner	Section	Contains	Type	Access	Initial	Mapped as
00380000	00001000				Priv	RW	RW	
00390000	00001000				Priv	RW	RW	
003A0000	00002000				Priv	RWE	RWE	
003B0000	00001000				Map	RW	RW	
00400000	00001000	UnPackM		PE header	Imag	R	RWE	
00401000	0006C000	UnPackM	hmimys	code	Imag	R	RWE	
0046D000	0002F000	UnPackM	hmimys	SFX, imports,	Imag	R	RWE	
0049C000	00001000	UnPackM	hmimys		Imag	R	RWE	
004A0000	00006000				Map	R E	R E	
00560000	00002000				Map	R E	R E	
00570000	00103000				Map	R	R	

Buscamos el final del loop desencreiptador y le ponemos un Bp con F2:

Address	Hex dump	Disassembly	Comment
00475067	8B45 08	mov eax, [ebp+8]	
0047506A	3B45 1C	cmp eax, [ebp+1C]	
0047506D	^ 0F82 74FDFFFF	jb 00474DE7	
00475073	8B45 C8	mov eax, [ebp-38]	UnPackMe.0049B4B7
00475076	8B4D 14	mov ecx, [ebp+14]	
00475079	2B45 0C	sub eax, [ebp+C]	
0047507C	8901	mov [ecx], eax	
0047507E	8B45 20	mov eax, [ebp+20]	
00475081	8B4D 08	mov ecx, [ebp+8]	
00475084	8908	mov [eax], ecx	
00475086	33C0	xor eax, eax	
00475088	5F	pop edi	
00475089	5E	pop esi	
0047508A	5B	pop ebx	
0047508B	C9	leave	
0047508C	C2 1C00	retn 1C	

Pasamos el "RETN 1C" quitando el BP por su puesto no vaya a ser que nos pesquen por ahí ;) y casualmente caemos aquí:

Address	Hex dump	Disassembly	Comment
00475109	68 00800000	push 8000	
0047510E	6A 00	push 0	
00475110	FF75 FC	push dword ptr [ebp-4]	
00475113	8BF0	mov esi, eax	
00475115	FF55 20	call [ebp+20]	kernel32.VirtualFree
00475118	85F6	test esi, esi	
0047511A	v 74 05	je short 00475121	

Vaya por donde, ¿no será que ya terminó su trabajo? Jeje.. pronto debería saltar al OEP xD y luego empunja el inicio de la sección antes de un call, o sea que se le pasa ese valor como uno de los 2 argumentos que recibe, el otro lo vemos en la pila:

Address	Value	Comment
0012FF80	00401000	UnPackMe.00401000
0012FF84	0006C000	← tamaño sección .CODE
0012FF88	004747F1	UnPackMe.004747F1
0012FF8C	7FFDF000	
0012FF90	00000003	
0012FF94	00000000	
0012FF98	00000000	
0012FF9C	00140000	
0012FFA0	0012FFF0	
0012FFA4	004748B1	RETURN to UnPackMe.004748B1 from UnPackMe.00475098

Que casualidad que es el mismo tamaño de esta sección, jeje.. entro y veo que busca los CALLs y JMPs tipo #E9# dentro de la sección de código:

Address	Hex dump	Disassembly		Comment
00474922	8A0C07	mov	cl, [edi+eax]	lee de .CODE
00474925	80F9 E8	cmp	cl, 0E8	es un CALL?
00474928	74 0B	je	short 00474935	
0047492A	80F9 E9	cmp	cl, 0E9	es un JMP?
0047492D	74 06	je	short 00474935	
0047492F	40	inc	eax	
00474930	E9 42010000	jmp	00474A77	
00474935	8BC8	mov	ecx, eax	; si es CALL o un
00474937	2B4D F8	sub	ecx, [ebp-8]	; JMP salta aqui!
0047493A	8945 F8	mov	[ebp-8], eax	

Al retornar del call deja en EAX un puntero EAX==46B000 donde hay:

Address	Hex dump	ASCII
0046B000	8C 0E 46 00 57 49 4E 4D 4D 2E 64 6C 6C 00 50 6C	?F.WINMM.dll.Pl
0046B010	61 79 53 6F 75 6E 64 41 00 00 74 09 46 00 4B 45	aySoundA..t.F.KE
0046B020	52 4E 45 4C 33 32 2E 64 6C 6C 00 47 65 74 53 79	RNEL32.dll.GetSy
0046B030	73 74 65 6D 54 69 6D 65 00 47 65 74 4C 6F 63 61	stemTime.GetLoca
0046B040	6C 54 69 6D 65 00 47 65 74 46 69 6C 65 54 79 70	lTime.GetFileTyp
0046B050	65 00 47 65 74 53 74 61 72 74 75 70 49 6E 66 6F	e.GetStartupInfo
0046B060	41 00 47 65 74 43 6F 6D 6D 61 6E 64 4C 69 6E 65	A.GetCommandLine
0046B070	41 00 52 74 6C 55 6E 77 69 6E 64 00 52 61 69 73	A.RtlUnwind.Rais

Que tiene toda la pinta de ser las apis que importa el cracke o unpackme que tratamos, después carga la DLL que necesita:

Address	Hex dump	Disassembly		Comment
004748BE	8BF8	mov	edi, eax	
004748C0	56	push	esi	
004748C1	FF13	call	[ebx]	kernel32.LoadLibraryA
004748C3	8BE8	mov	ebp, eax	
004748C5	AC	lods	byte ptr [esi]	

Y por fin busca la dirección de la API tratada "PlaySoundA" justo después en esta llamada a GPAddress:

Address	Hex dump	Disassembly		Comment
004748D8	83EE 04	sub	esi, 4	
004748DB	56	push	esi	
004748DC	55	push	ebp	
004748DD	FF53 04	call	[ebx+4]	kernel32.GetProcAddress
004748E0	AB	stos	dword ptr es:[edi]	salva la API en la IAT
004748E1	EB E2	jmp	short 004748C5	

Si nos fijamos la api que obtuvo la guardó en la IAT:

Address	Hex dump	ASCII
00460E8C	62 8B B0 76 00 00 00 00 16 29 06 00 80 29 06 00	b嫌v....) .€)
00460E9C	70 29 06 00 5E 29 06 00 4E 29 06 00 3A 29 06 00	p) .^) .N) .:
00460EAC	26 29 06 00 [illegible] 00 00 00 E0 28 06 00	&) .) .....?
00460EBC	D4 28 06 00 [illegible] 28 06 00 A0 28 06 00	? .? .? .? .
00460ECC	8C 28 06 00 62 28 06 00 80 28 06 00 70 28 06 00	? .b(.€(.p(
00460EDC	00 00 00 00 0C 2B 06 00 FA 2A 06 00 E8 2A 06 00	.....+ .? .? .
00460EEC	1E 2B 06 00 D8 2A 06 00 C6 2A 06 00 AE 2A 06 00	+ .? .? .?

API salvada¡!!!!!

Ya sabemos como obtiene algunas apis , veremos si las hace todas igual o varia de método, la siguiente el kernel32.GetSystemTime y lo hace de la misma manera, como vemos que no varía nunca ponemos un BP al final de la rutina:

Address	Hex dump	Disassembly		Comment
004748E9	55	push	ebp	
004748EA	FF53 04	call	[ebx+4]	
004748ED	AB	stos	dword ptr es:[edi]	
004748EE	^ EB DA	jmp	short 004748CA	
004748F0	83EB 44	sub	ebx, 44	fin RUTINA
004748F3	8BF3	mov	esi, ebx	
004748F5	AD	lods	dword ptr [esi]	
004748F6	50	push	eax	
004748F7	C3	retn		

Cuando para tenemos TODA la IAT limpita, jeje:

Address	Hex dump				ASCII
00460824	B1 63 DB 77	10 24 DA 77	00 00 00 00	B1 38 31 77	.c.w.$.w.....81w
00460834	3D 51 31 77	00 00 00 00	02 3D C4 77	00 42 C4 77	=Q1w.....=.w.B.w
00460844	45 29 C4 77	9D 50 C4 77	10 85 C4 77	7E 4F C4 77	E).w.P.w...w~O.w
00460854	BC 6C C4 77	[illegible]	[illegible]	[illegible] D C6 77	.l.wP3.w.2.w...w
00460864	B2 92 C6 77	[illegible]	[illegible]	[illegible] 2 C4 77	...w...w.~.w/R.w
00460874	45 92 C4 77	[illegible]	[illegible]	[illegible] 9 C4 77	E..w.p.wu..w...w
00460884	A2 43 C5 77	[illegible]	[illegible]	[illegible] 1 C4 77	.C.w.Z.w...w.!.w
00460894	EE 63 C4 77	[illegible]	[illegible]	[illegible] 3 C4 77	.c.w...w.6.wT..w
004608A4	22 4F C4 77	18 B6 C4 77	[illegible]	80 C4 77 A9 2B C4 77	"O.w...wv..w?.w
004608B4	A2 42 C5 77	12 95 C6 [illegible]	[illegible]	3E 3C C4 77 B4 DC C6 77	.B.w...wn<.w...w
004608C4	9D 88 C4 77	C6 83 C4 [illegible]	[illegible]	E0 20 C4 77 98 1B C4 77	...w...w. .w...w
004608D4	68 4E C4 77	F8 A9 C[illegible]77	[illegible]	3C 65 C4 77 C0 C8 C5 77	hN.w.?w<e.w...w
004608E4	F9 AA C4 77	75 74 [illegible] 77	[illegible]	EA 34 C4 77 FE 93 C4 77	...wut.w.4.w...w
004608F4	EB 1E C4 77	F5 54 [illegible]C4 77	[illegible]	6A 54 C4 77 78 23 C4 77	...w.T.wjT.wx#.w
00460904	1A 1E C4 77	6F 1[illegible] C4 77	[illegible]	11 50 C4 77 E9 35 C5 77	...wo..w.P.w.5.w
00460914	2A 59 C5 77	7C C6 C5 77	[illegible]	CD E2 C6 77 2B A0 C5 77	*Y.w|..w...w+..w
00460924	7C 21 C4 77	33 83 C4 77	[illegible]	F4 69 C7 77 5F 3A C4 77	|!.w3..w.i.w_:.w
00460934	07 36 C4 77	8E 4E C4 77	[illegible]	5C 85 C4 77 C4 5D C4 77	.6.w.N.w\..w.].w
00460944	46 D8 C4 77	82 3F C4 77	[illegible]	2B 53 C4 77 71 4B C4 77	F..w.?.w+S.wqK.w
00460954	16 4D C5 77	3B 48 C5 77	[illegible]	0B 46 C5 77 2B 47 C5 77	.M.w;H.w.F.w+G.w
00460964	3D 51 C4 77	9F 86 C4 77	[illegible]	9B 40 C4 77 00 00 00 00	=Q.w...w.@.w....

Ya tenemos TODA la IAT bien limpinta;)

Address	Hex dump	Disassembly		Comment
004748E0	AB	stos	dword ptr es:[edi]	salva la API en la IAT
004748E1	^ EB E2	jmp	short 004748C5	
004748E3	25 FFFFFF7F	and	eax, 7FFFFFFF	
004748E8	50	push	eax	
004748E9	55	push	ebp	
004748EA	FF53 04	call	[ebx+4]	
004748ED	AB	stos	dword ptr es:[edi]	
004748EE	^ EB DA	jmp	short 004748CA	
004748F0	83EB 44	sub	ebx, 44	fin RUTINA
004748F3	8BF3	mov	esi, ebx	
004748F5	AD	lods	dword ptr [esi]	
004748F6	50	push	eax	
004748F7	C3	retn		ESTE!!!! ←
004748F8	55	push	ebp	
004748F9	8BEC	mov	ebp, esp	
004748FB	83EC 10	sub	esp, 10	

Y si traceamos hasta el siguiente RET

Y lo superamos quedamos detenidos aquí ¿?

Solución al Concurso 106 nivel 3 - [SyXe'05]

Address	Hex dump	Disassembly		Comment
004271B0	55	db	55	CHAR 'U'
004271B1	8B	db	8B	
004271B2	EC	db	EC	
004271B3	6A	db	6A	CHAR 'j'
004271B4	FF	db	FF	
004271B5	68	db	68	CHAR 'h'
004271B6	60	db	60	CHAR '`'
004271B7	0E	db	0E	
004271B8	45	db	45	CHAR 'E'
004271B9	00	db	00	
004271BA	68	db	68	CHAR 'h'
004271BB	C8	db	C8	
004271BC	92	db	92	
004271BD	42	db	42	CHAR 'B'
004271BE	00	db	00	
004271BF	64	db	64	CHAR 'd'
004271C0	A1	db	A1	
004271C1	00	db	00	
004271C2	00	db	00	

¿Qué es esto? Se preguntarán algunos, pues es que Olly no analizó todavía esta parte, o bien cuando lo hizo eran todavía datos inservibles, hagamos un REMOVE ANALISIS y ya veremos que cambio ;)

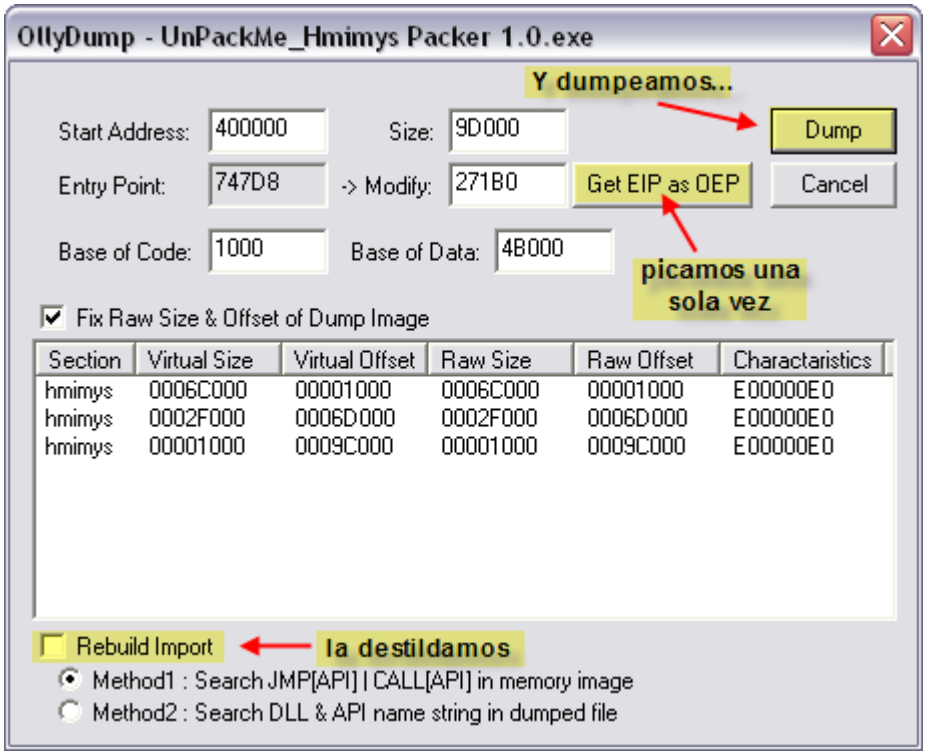
Address	Hex dump	Disassembly		Comment
004271B0	55	push	ebp	
004271B1	8BEC	mov	ebp, esp	
004271B3	6A FF	push	-1	
004271B5	68 600E4500	push	00450E60	
004271BA	68 C8924200	push	004292C8	
004271BF	64:A1 00000000	mov	eax, fs:[0]	
004271C5	50	push	eax	
004271C6	64:8925 0000000	mov	fs:[0], esp	
004271CD	83C4 A8	add	esp, -58	
004271D0	53	push	ebx	
004271D1	56	push	esi	

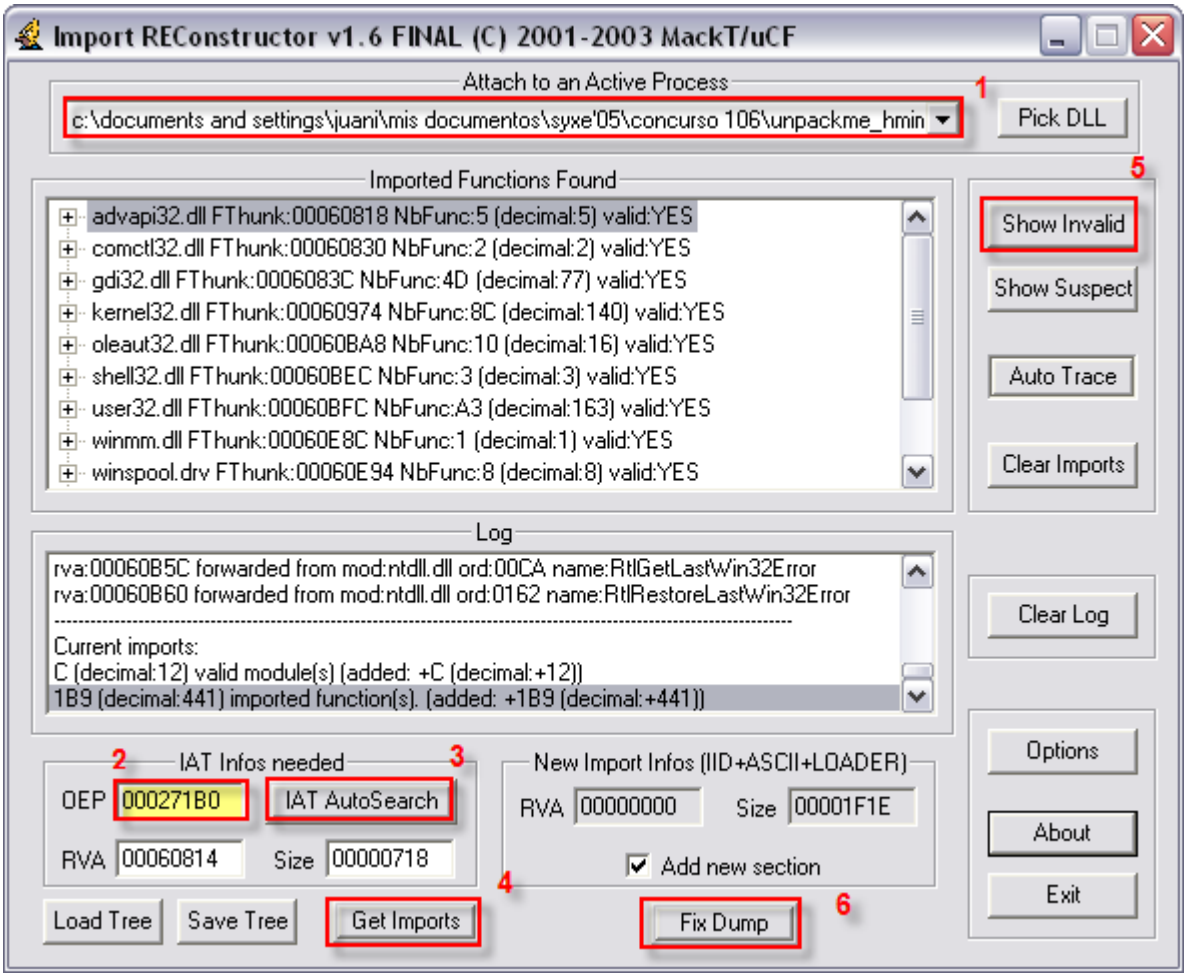
El OEP!!!

Anotemos este dato:

OEP: 4271B0

Añprovechamos para dumpear con el propio plugin del Olly (Ollydump)

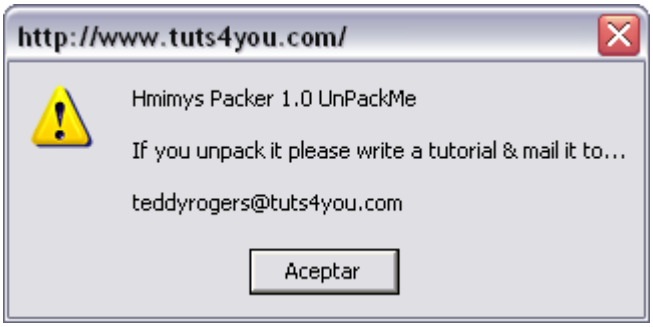




Y como la IAT está intacta con el ImpREC dejamos un desempakado como merece:

Una vez hecho esto ejecutamos tute_.exe (si es que llamamos tute.exe al dumpeado) y nos aparecerá el esperado cartel ;)

Solución al Concurso 106 nivel 3 - [SyXe'05]



Y con esto terminamos el tute del con curso 106 nivel 3, espero que os haya gustado y que sirva de ayuda a alguien con el tema de desempakado, se que no es gran cosa sólo pero con eso me conformaría, un abrazo a todo el grupo y decir que mi vuelta es ya un hecho, espero escribir muchos tutes más. Un abrazo a TODOS¡!!!!

PD: No tengo PC pues cascó ;(pero pronto lo tendre de nuevo, jeje..

Para cualquier cosa ya sabéis donde contactarme: syxe05@gmail.com XD

visita obligada a:

HYPERLINK "http://storage2.ricardonarvaja.com.ar/web/"http://storage2.ricardonarvaja.com.ar/we